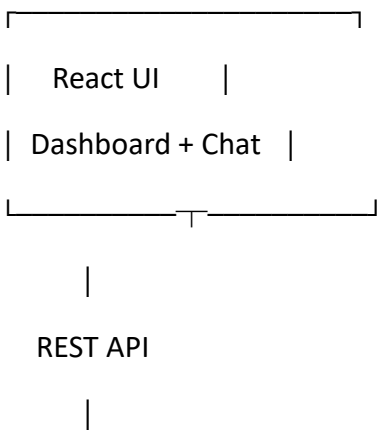
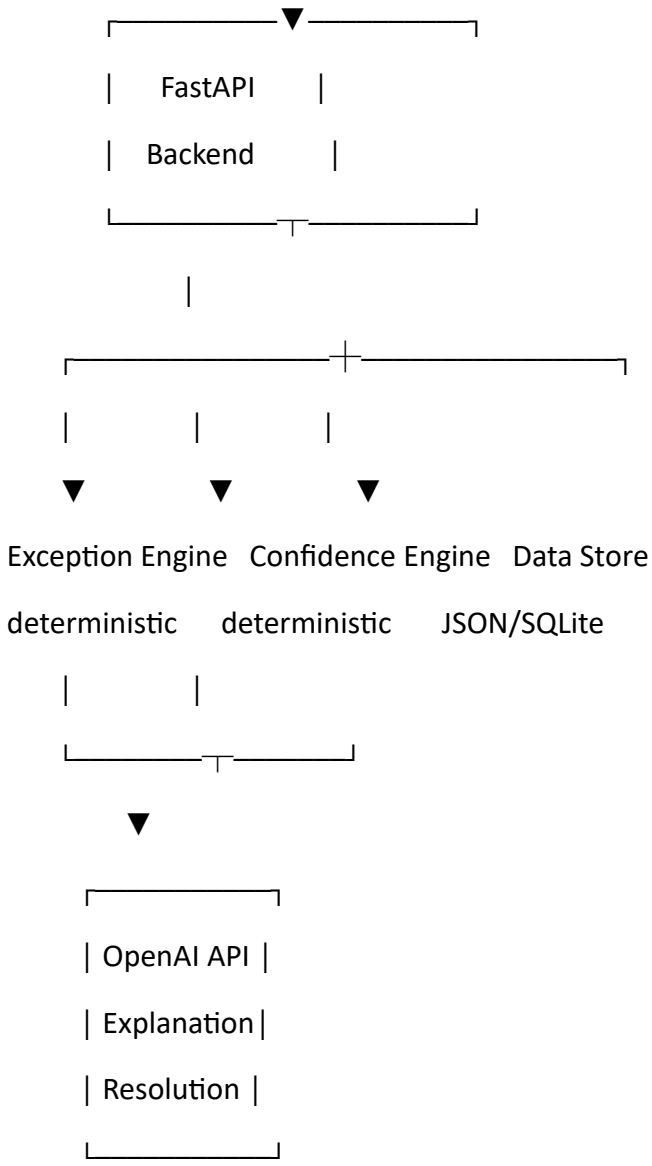
Recommended tech stack

Layer	Technology	Why
Frontend	**React + Vite**	Fast to build, professional UI
Styling	**Tailwind CSS**	Very quick dashboard styling
Icons	**Lucide React**	Clean enterprise look
Backend	**Python + FastAPI**	Simple APIs + excellent AI/data ecosystem
Data	**JSON / SQLite**	No database setup headache
AI	**OpenAI API**	Explanation + resolution recommendation
AI output	**Structured JSON**	Prevents unreliable free-form responses
Business logic	**Python deterministic rules**	Confidence thresholds shouldn't depend on LLM
Charts	**Recharts**	Simple exception metrics
HTTP	**Axios/fetch**	Frontend ↔ backend
Deployment	**Vercel + Render/Railway** or local demo	Easy sharing
Version control	**GitHub**	Required repository submission

Architecture





This separation is **very important**.

---

1. React + Vite for the frontend

I'd use React rather than building the entire thing in Python/Streamlit.

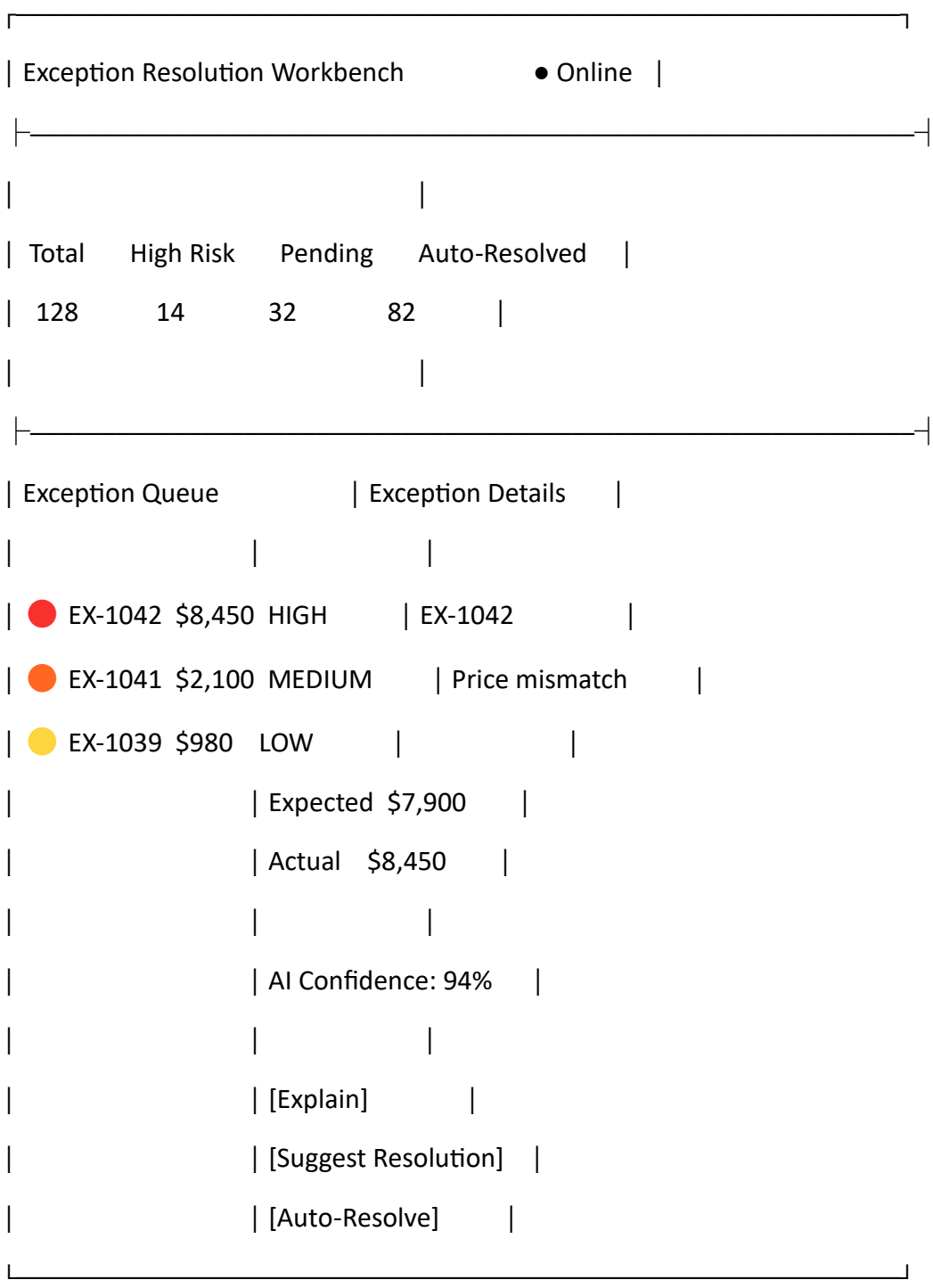
Why?

Because the assignment asks for a **lightweight web app — a chatbot paired with a simple dashboard**.

A React dashboard will let us make it look much more like a real enterprise product.

We can have:

Dashboard



That's going to be much more convincing in a 5-minute demo.

---

2. FastAPI backend

Use:

Python + FastAPI

The backend handles:

GET /exceptions

GET /exceptions/{id}

POST /exceptions/{id}/explain

POST /exceptions/{id}/resolve

POST /exceptions/{id}/auto-resolve

GET /metrics

We don't need a complicated microservice architecture.

One FastAPI application is enough.

---

3. JSON initially, SQLite optionally

Don't start with PostgreSQL.

We don't need it.

For the assessment, synthetic data is explicitly allowed.

We can create:

data/

  exceptions.json

  transactions.json

  resolutions.json

Example:

```
{
  "id": "EX-1042",
  "vendor": "ABC Supplies",
```

"po_number": "PO-8841",

"invoice_amount": 8450,

"expected_amount": 7900,

"exception_type": "PRICE_MISMATCH",

"severity": "HIGH",

"confidence": 0.94,

"status": "PENDING"

}

This makes development extremely fast.

If we later want persistence, we can move to SQLite without changing the overall architecture.

---

4. Most important: deterministic Exception Engine

Do NOT let the LLM decide everything.

This is probably one of the biggest things that can differentiate our submission.

For example:

Expected amount = ₹7,900

Actual amount = ₹8,450

Difference = ₹550

Difference % = 6.96%

→ PRICE_MISMATCH

→ HIGH severity

This is normal Python logic.

Similarly:

quantity mismatch

amount mismatch

duplicate transaction

missing reference

unusual amount

can all be deterministic.

Then the LLM gets the structured exception and explains it.

Architecture becomes:

Raw Transaction

↓

Deterministic Exception Engine

↓

Structured Exception

↓

LLM

↓

Explanation / Recommendation

That is much safer than:

Raw Transaction

↓

"Hey GPT, figure out what's wrong"

↓

🤷

---

5. Confidence Engine

This is where I think we can score particularly well.

The assessment specifically requires:

"Define and enforce a confidence threshold for auto-resolution."

So we should create an explicit policy.

For example:

Confidence >= 0.90

↓

AUTO-RESOLVE

0.70 – 0.89

↓

SUGGEST + HUMAN APPROVAL

< 0.70

↓

HUMAN REVIEW

But here's an important design decision:

The LLM should NOT directly control the threshold.

Instead:

```
if confidence >= 0.90:
    action = "AUTO_RESOLVE"
elif confidence >= 0.70:
    action = "SUGGEST"
else:
    action = "HUMAN_REVIEW"
```

Then we can explain this in the demo.

That demonstrates **controlled AI**, rather than "LLM does everything."

---

6. OpenAI API

Use the LLM for two things:

A. Explanation

User:

Why was EX-1042 flagged?

LLM receives structured facts:


```
{
  "exception_type": "PRICE_MISMATCH",
  "expected": 7900,
  "actual": 8450,
  "difference": 550,
  "vendor": "ABC Supplies"
}
```


And produces:

EX-1042 was flagged because the invoice amount

is ₹550 higher than the purchase order.

The PO specifies ₹7,900, while the invoice is

₹8,450.

The quantity matches, so the exception is isolated

to the unit price.

B. Suggested resolution

For example:

"Request vendor correction for the unit price and place the invoice on hold."

Again, the LLM is **reasoning over structured facts**, rather than inventing facts.

---

7. Structured LLM responses

This is another thing I'd strongly recommend.

Instead of asking the model for:

Give me an explanation.

we make it return something like:

```
{
  "explanation": "...",
  "root_cause": "...",
  "suggested_action": "...",
  "confidence": 0.94,
  "evidence": [
    "Invoice amount: ₹8,450",
    "PO amount: ₹7,900"
  ]
}
```

Then our frontend can display those fields separately.

That gives us a very nice UI:

AI ANALYSIS

Root Cause

Price mismatch

Evidence

✓ Invoice: ₹8,450

✓ PO: ₹7,900

✓ Difference: ₹550

Suggested Resolution

Request vendor correction

AI Confidence

94%

Decision

AUTO-RESOLVE ELIGIBLE

Much more professional.

---

8. Audit trail

We absolutely should include this.

Every action should generate something like:

14:32:08 EX-1042 created

14:32:11 Exception classified

14:32:14 AI explanation generated

14:32:18 Resolution suggested

14:32:27 Reviewer approved

14:32:29 Exception resolved

This gives us an **audit trail**, which is very appropriate for an enterprise workflow.

---

9. Chat interface

Don't build a huge ChatGPT clone.

Just make a contextual assistant.

When the reviewer selects:

EX-1042

the right panel contains:

Ask about this exception...

> Why was this flagged?

> What should I do?

> Show the evidence.

> Can this be auto-resolved?

This is enough.

The assessment specifically asks the reviewer to be able to ask the AI Employee to **explain, suggest a resolution, or auto-resolve**.

---

10. UI libraries

I'd keep these lightweight:

Tailwind CSS

For layout.

Lucide React

For icons.

Recharts

For a couple of charts.

For example:

Exception distribution

Price mismatch [redacted]

Quantity mismatch [redacted]

Duplicate [redacted]

Missing PO [redacted]

and:

Resolution status

Auto-resolved 64%

Human approved 21%

Pending 10%

Escalated 5%

Don't overdo charts. This isn't a BI assignment.

What I DON'T recommend

❌ **LangChain**

We don't need it.

It can make the architecture look more complicated without adding much value for this problem.

❌ **LangGraph**

Same reason.

We don't need a complex agent graph.

❌ **Pinecone / vector DB**

No need.

We're not building a RAG system.

❌ **PostgreSQL**

Not necessary for the assessment.

❌ **Docker**

Optional, but I wouldn't spend our initial time on it.

❌ **Kubernetes / microservices**

Absolutely unnecessary.

❌ **Multiple agents**

That's Problem #5. We're doing #9.

❌ **Authentication**

Not needed unless we have extra time.

Final stack I'd lock in

FRONTEND

React

Vite

Tailwind CSS

Lucide React

Recharts

BACKEND

Python

FastAPI

Pydantic

DATA

JSON initially

SQLite if persistence becomes useful

AI

OpenAI API

Structured JSON responses

LOGIC

Python deterministic exception engine

Python confidence/approval engine

TOOLS

Git

GitHub

VS Code

And the philosophy should be:

LLM = intelligence

Python = control

React = experience

JSON/SQLite = state

That's a very clean architecture for an FDE assessment.

Most importantly, it directly maps to what Supervity says it will evaluate: the **human-in-command interaction loop, confidence-threshold logic, and overall usability**, rather than merely whether we can call an LLM.

One thing I would do before coding

Before touching React, we should define **the exact business scenario and synthetic data**. That will determine what the exception engine, AI prompts, confidence scores, UI, and demo all look like.

I suggest we build it around **invoice/payment exceptions**, because that gives us the strongest enterprise story while keeping the data and logic simple.